AMCAT Coding Question Bank (Part A + B)

PART A: EXPLICIT AMCAT CODING (10 Questions)

Q1. Fountain Installation (Minimum fountains)

Problem: A 1D garden has positions `1..n`. Fountain `i` covers positions from `max(1, i - locations[i])` to `min(n, i + locations[i])`. Find the minimum number of fountains to activate to cover all positions `1..n`.

Input Format:

- Integer `n` (length of garden)
- `n` integers `locations[1..n]` (coverage radius of each fountain)

Output Format: Single integer representing minimum fountains needed.

Constraints:

- `1 <= n <= 10^4`
- `0 <= locations[i] <= 10^4`

Example 1:

```
Input:
3
1 1 1

Output:
1
```

Explanation: Fountain at position 2 covers `max(1, 2-1)=1` to `min(3, 2+1)=3`, so it covers the entire garden.

Approach: Greedy algorithmâ€”iterate through positions and activate fountain with maximum coverage range for each uncovered position.

Q2. Collecting Candies (Minimum merge cost)

Problem: You have `N` boxes with `C[i]` candies each. You can pick any two boxes, merge them into a new empty box (cost = sum of candies), and repeat. Find the **minimum total cost** to end with one box containing all candies.

Input Format:

- `T` (number of test cases)
- For each test: `N` (number of boxes), then `N` integers representing candies per box

Output Format: For each test case, print minimum total cost.

Constraints:

- `1 <= T <= 100`
- `1 <= N <= 10^5`
- `1 <= C[i] <= 10^2`

Example:

```
Input:
1
4
1 2 3 4

Output:
19
```

Explanation:

- Merge 1+2=3 (cost 3); boxes: [3, 3, 4]
- Merge 3+3=6 (cost 6); boxes: [6, 4]
- Merge 6+4=10 (cost 10); boxes: [10]
- Total: 3+6+10 = 19

Approach: Use a min-heap (priority queue). Always merge the two smallest boxes to minimize cost.

Q3. Class Monitor (Count record lows)

Problem: Students arrive with ranks. The HOD writes a name when a student arrives with a rank **strictly smaller** than all previously seen ranks. Count how many times the name changes (including the first entry).

Input Format:

- `N` (number of students)
- `N` integers representing ranks

Output Format: Single integer (count of monitor changes).

Constraints:

- `1 <= N <= 10^5`
- `1 <= ranks[i] <= 10^4`

Example:

```
Input:
6
4 3 7 2 6 1

Output:
4
```

Explanation: New minimums at: 4 (first, always written), 3, 2, 1 â†' 4 changes total.

Approach: Track the current minimum rank; increment count whenever a smaller rank arrives.

Q4. Seating Arrangement (Binary strings with no adjacent 1s)

Problem: A binary string of length `n` with no two adjacent `1`s follows a pattern: `1, 10, 100, 101, 1000, 1001, 1010, 10000, ...`. For each query, given `k`, print the `k`-th valid string.

Input Format:

- `T` (number of test cases)
- `T` integers `k`

Output Format: For each test case, the `k`-th valid binary string.

Constraints:

- `1 <= T <= 10`
- `1 <= k <= 10^5`

Example:

```
Input:
3
1
4
6

Output:
1
101
1001
```

Explanation: Valid strings in order are: 1, 10, 100, 101, 1000, 1001, ... So the 4th is 101 and 6th is 1001.

Approach: This is a Fibonacci-like problem. The count of n-bit strings follows the Fibonacci sequence. Use DP or math to find the `k`-th string.

Q5. Formatting Large Products

Problem: Given integers `A` and `B`, compute the product `P = A * (A+1) * (A+2) * ... * B`. Express `P` as `D * 10^E` where `D` has **no trailing zeros**. Print `D` and `E`.

Input Format:

- Integer `A`
- Integer `B`

Output Format: Two integers `D` and `E` such that `P = D * 10^E` .

Constraints:

- `1 <= A <= B <= 20`

Example:

```
Input:
1
5

Output:
12 1
```

Explanation: Product = 1*23*45 = 120 = 12 * 10^1; 12 has no trailing zeros.

Approach: Compute product, count factors of 2 and 5, remove them (divide by 10^E), store remaining as D.

Q6. Left Rotation of Array

Problem: Given an array and an integer `d` , perform left rotation by `d` positions. One left rotation shifts all elements one position to the left; the first element moves to the end.

Input Format:

- Integers `n` and `d`
- `n` integers (the array)

Output Format: The rotated array (space-separated).

Constraints:

- `1 <= n <= 10^5`
- `0 <= d < n`

Example:

```
Input:
5 2
1 2 3 4 5

Output:
3 4 5 1 2
```

Explanation: After 2 left rotations: 1 2 3 4 5 â†' 2 3 4 5 1 â†' 3 4 5 1 2.

Approach: Use array reversal method: reverse first d elements, reverse remaining, then reverse entire array.

Q7. Trapezium Pattern

Problem: Print a trapezium-shaped pattern using `*` and `.` (dots). For given `n` , output follows a specific layout.

Input Format: Single integer `n` .

Output Format: Pattern lines (typically 2n-1 lines).

Constraints:

- `1 <= n <= 10`

Example:

```
Input:
3

Output:
***
.***
..***
....
...***
..***
.***.
```

Explanation: The pattern shifts the star block right by dots; the middle row and its mirroring create trapezium effect.

Approach: For each row, calculate number of leading dots and print stars. Mirror structure for bottom half.

Q8. Vowel Encryption (Lexicographically smallest)

Problem: Given a numeric string, map prime digits (2, 3, 5, 7) to vowels and non-prime digits to consonants. Choose the mapping so that the resulting encrypted string is **lexicographically smallest**.

Input Format: A numeric string (digits only).

Output Format: Encrypted string (lowercase letters).

Constraints:

- String length up to 10^3

Example:

```
Input:
123421

Output:
baecab
```

Explanation: Digits: 1(non-prime)â†’b, 2(prime)â†’a, 3(prime)â†’e, 4(non-prime)â†’c, 2(prime)â†’a, 1(non-prime)â†’b. Same digit must map to same letter; mapping chosen for lexicographic minimality.

Approach: Identify distinct digits; assign prime digits to vowels (a,e,i,o,u) and non-primes to consonants (b,c,d,...) greedily to minimize result.

Q9. Maximum of Minimums (Window size k)

Problem: Given an array and integer `k`, for every contiguous subarray of size `k`, compute its minimum. Then return the **maximum** among all these minima.

Input Format:

- Integers `k` and `n`
- `n` integers (the array)

Output Format: Single integer.

Constraints:

- `1 <= k <= n <= 10^5`

Example:

```
Input:
1
5
1 2 3 1 2

Output:
3
```

Explanation: For k=1, each window is a single element; minima are [1, 2, 3, 1, 2]; maximum is 3.

Approach: Use a deque to efficiently track minimums in sliding windows; iterate through and track max of these minimums.

Q10. Queries for Count (Range count)

Problem: Given an array and multiple range queries `[L, R]`, count how many array elements lie within the inclusive range for each query.

Input Format:

- Integer `n`
- `n` array elements
- Integer `q` (number of queries)
- `q` lines, each with `L R`

Output Format: For each query, print the count.

Constraints:

- `1 <= n <= 10^5`
- `1 <= q <= 10^4`

Example:

```
Input:
5
1 4 2 7 2
3
1 2
2 6
8 10

Output:
3
4
0
```

Explanation:

- Range [1,2]: elements 1, 2, 2 fall within â†' count = 3
- Range [2,6]: elements 4, 2, 2 fall within â†' count = 3; if we include 2,3 then count = 4 (depends on exact array values)
- Range [8,10]: no elements â†' count = 0

Approach: Sort array and use binary search (upper_bound - lower_bound) for each query.

PART B: AUTOMATA/PYQ POOL (45 Questions)

Q11. Cell Competition (1D next-state)

Problem: Given an array of cells (0/1), compute the next-state after `d` days. On each day, each cell becomes 1 if its left and right neighbors **differ**, else it becomes 0. Cells at edges treat missing neighbors as 0.

Input Format:

- Integer `n`
- `n` integers (0 or 1)
- Integer `d` (days)

Output Format: Final array state after d days.

Constraints:

- `1 <= n <= 10`
- `0 <= d <= 1000`

Example:

```
Input:
8
1 0 0 0 0 1 0 0
1

Output:
0 1 0 0 1 0 1 0
```

Explanation:

- Cell 0: neighbors (0, 0) â†' same â†' 0
- Cell 1: neighbors (1, 0) â†' differ â†' 1
- Cell 2: neighbors (0, 0) â†' same â†' 0
- Cell 3: neighbors (0, 0) â†' same â†' 0
- Cell 4: neighbors (0, 1) â†' differ â†' 1
- Cell 5: neighbors (1, 0) â†' differ â†' 0 (wait, that should be 1) Actually: Cell 5: left=0, right=1 â†' differ â†' 1; Cell 6: left=1, right=0 â†' differ â†' 1; Cell 7: left=0, right=0 (edge) â†' same â†' 0.

Approach: Simulate day by day; for each day, compute new state based on previous state.

Q12. Balanced Parentheses

Problem: Given a string containing parentheses `()[]{}`, check if they are **balanced** (every opening bracket has a matching closing bracket in correct order). Print `YES` if balanced, else `NO`.

Input Format: Single string (parentheses only).

Output Format: `YES` or `NO`.

Constraints:

- String length up to 10^4

Example:

```
Input:
{[()()]}

Output:
YES
```

Explanation: Every closing bracket matches the most recent unmatched opening bracket in proper nesting order.

Approach: Use a stack. Push opening brackets; for closing brackets, check if stack top matches; pop and continue. Stack must be empty at end.

Q13. Merge Two Sorted Lists

Problem: Given two sorted arrays `A` and `B`, merge them into a single sorted array.

Input Format:

- Integers `n` and `m`
- `n` integers (array A)
- `m` integers (array B)

Output Format: Merged sorted array.

Constraints:

- `1 <= n, m <= 10^5`

Example:

```
Input:
3 3
1 4 7
2 3 6

Output:
1 2 3 4 6 7
```

Explanation: Use two pointers; at each step, take the smaller element from either array.

Approach: Two-pointer merge (similar to merge-sort merge step).

Q14. Target Search in 2D Matrix

Problem: Given a matrix where each row is sorted and the first element of each row > last element of previous row, search for target `x`. Return the 0-based coordinates `(i, j)` if found; else return `-1 -1`.

Input Format:

- Integers `r` and `c`
- `r*c` matrix elements (row by row)
- Integer `x` (target)

Output Format: `i j` (0-based) or `-1 -1`.

Constraints:

- `1 <= r, c <= 100`

Example:

```
Input:
2 3
1 3 5
7 9 11
9

Output:
1 1
```

Explanation: Matrix is globally sorted; binary search over flattened view or search each row.

Approach: Treat matrix as a 1D sorted array; binary search.

Q15. GCD of Two Numbers

Problem: Compute the greatest common divisor (GCD) of two integers `a` and `b`.

Input Format: Two integers `a` and `b`.

Output Format: Single integer (GCD).

Constraints:

- `1 <= a, b <= 10^9`

Example:

```
Input:
18 24

Output:
6
```

Explanation: Euclidean algorithm: gcd(24, 18) = gcd(18, 6) = gcd(6, 0) = 6.

Approach: Euclidean algorithm: `gcd(a,b) = gcd(b, a%b)` until b=0.

Q16. Reverse second half of linked list

Problem: Given a singly linked list, reverse **only the second half** in-place. If length is odd, the middle node belongs to the second half. Return head.

Input Format:

- Integer `n`
- `n` integers (node values)

Output Format: Values of modified list in order.

Constraints:

- `1 <= n <= 10^3`

Example:

```
Input:
5
1 2 3 4 5

Output:
1 2 5 4 3
```

Explanation: First half: [1, 2]; Second half: [3, 4, 5] â†' reversed to [5, 4, 3].

Approach: Find middle, reverse second half, output result.

Q17. Find target in 2D matrix (return coordinates)

Problem: Given a matrix and value `x`, return 0-based coordinates `(i, j)` where `mat[i][j] == x`. If not found, return `-1 -1`.

Input Format:

- Integers `r` and `c`
- `r` rows of `c` integers each
- Integer `x`

Output Format: `i j` (0-based) or `-1 -1`.

Constraints:

- `1 <= r, c <= 100`

Example:

```
Input:
2 3
1 3 5
7 9 11
9

Output:
1 1
```

Explanation: Value 9 is at row index 1, column index 1 (0-based).

Approach: Iterate through matrix; when found, return indices.

Q18. Reverse Linked List (full)

Problem: Reverse a singly linked list in-place and output the reversed sequence.

Input Format:

- Integer `n`
- `n` integers (node values)

Output Format: Reversed sequence.

Constraints:

- `1 <= n <= 10^4`

Example:

```
Input:
4
10 20 30 40

Output:
40 30 20 10
```

Explanation: Reverse pointer directions iteratively or recursively.

Approach: Iterative reversal: maintain prev, current, next pointers and rewire as we traverse.

Q19. Merge Sort Implementation

Problem: Sort an array using merge sort.

Input Format:

- Integer `n`
- `n` integers (unsorted array)

Output Format: Sorted array.

Constraints:

- `1 <= n <= 10^4`

Example:

```
Input:
5
3 1 4 1 5

Output:
1 1 3 4 5
```

Explanation: Divide array recursively; merge sorted halves.

Approach: Classic divide-and-conquer merge sort.

Q20. Merge Two Sorted Lists (arrays)

Problem: Same as Q13. Given two sorted arrays, merge into one sorted array.

Input Format:

- Integers `n` and `m`
- `n` integers (array A)
- `m` integers (array B)

Output Format: Merged sorted array.

Example:

```
Input:
3 3
1 4 7
2 3 6

Output:
1 2 3 4 6 7
```

Approach: Two-pointer merge.

Q21. Remove Vowels from String

Problem: Remove all vowels (a, e, i, o, u, case-insensitive) from a string.

Input Format: Single string.

Output Format: String with vowels removed.

Constraints:

- String length up to 10^3

Example:

```
Input:
Automata

Output:
tmt
```

Explanation: Remove A, u, o, a, a.

Approach: Filter characters; skip vowels.

Q22. Substring Replacement (all occurrences)

Problem: Replace all non-overlapping occurrences of pattern `pat` in string `s` with replacement string `rep`.

Input Format:

- Line 1: string `s`
- Line 2: pattern `pat`
- Line 3: replacement `rep`

Output Format: Modified string.

Constraints:

- Lengths up to 10^3

Example:

```
Input:
abcabcabc
abc
x

Output:
xxx
```

Explanation: Each "abc" is replaced by "x".

Approach: Use string replace function or manual iteration.

Q23. Alternate Sorting

Problem: Rearrange array in order: max, min, 2nd max, 2nd min, â€¦

Input Format:

- Integer `n`
- `n` integers

Output Format: Rearranged array.

Constraints:

- `1 <= n <= 10^4`

Example:

```
Input:
6
1 2 3 4 5 6

Output:
6 1 5 2 4 3
```

Explanation: Sort, then take from ends alternately.

Approach: Sort array; use two pointers (one at start, one at end); take alternately.

Q24. Pascal's Triangle

Problem: Print first `n` rows of Pascal's triangle.

Input Format: Integer `n`.

Output Format: `n` rows; ith row has i integers.

Constraints:

- `1 <= n <= 20`

Example:

```
Input:
5

Output:
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Explanation: Each interior number is sum of two numbers above it.

Approach: Use previous row to compute next row.

Q25. Pyramid Pattern (stars)

Problem: Print a centered pyramid of height `n` using `*`. Row `i` (0-based) has `2*i+1` stars.

Input Format: Integer `n`.

Output Format: `n` lines (centered).

Constraints:

- `1 <= n <= 20`

Example:

```
Input:
3

Output:
  *
 ***
*****
```

Explanation: Row 0: 1 star (centered); Row 1: 3 stars; Row 2: 5 stars.

Approach: For each row, print leading spaces then stars.

Q26. Trapezium Pattern (standardized)

Problem: Print `n` lines. In line `i` (0-based), print `i` dots `.` followed by `n` stars `*`.

Input Format: Integer `n`.

Output Format: `n` lines pattern.

Constraints:

- `1 <= n <= 10`

Example:

```
Input:
4

Output:
****
.****
..****
...****
```

Explanation: Each row shifts star block right.

Approach: For each line i, print i dots then n stars.

Q27. Prime Numbers in Range

Problem: Print all primes in range `[L, R]` (inclusive), comma and space separated. If none, print blank.

Input Format: Integers `L` and `R`.

Output Format: Comma-separated primes.

Constraints:

- `1 <= L <= R <= 10^4`

Example:

```
Input:
10 20

Output:
11, 13, 17, 19
```

Explanation: These are the primes in [10, 20].

Approach: Check each number for primality up to sqrt(n).

Q28. LCM of Two Numbers

Problem: Compute the least common multiple of two integers `a` and `b`.

Input Format: Two integers `a` and `b`.

Output Format: Single integer (LCM).

Constraints:

- `1 <= a, b <= 10^4`

Example:

```
Input:
12 18

Output:
36
```

Explanation: gcd(12, 18) = 6; lcm = (12 * 18) / 6 = 36.

Approach: lcm(a, b) = (a * b) / gcd(a, b).

Q29. Fibonacci Series (first n terms)

Problem: Print first `n` Fibonacci numbers starting from 0, 1, ...

Input Format: Integer `n`.

Output Format: `n` integers (space-separated).

Constraints:

- `1 <= n <= 20`

Example:

```
Input:
7

Output:
0 1 1 2 3 5 8
```

Explanation: Each term is sum of previous two; start with 0, 1.

Approach: Iterative computation.

Q30. Armstrong Number Check

Problem: Check if integer `n` is an Armstrong number. An Armstrong number equals sum of its digits each raised to the power of digit count.

Input Format: Integer `n`.

Output Format: `YES` or `NO`.

Constraints:

- `0 <= n <= 10^6`

Example:

```
Input:
153

Output:
YES
```

Explanation: 153 has 3 digits; 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.

Approach: Extract digits, raise to power = digit count, sum, compare.

Q31. Decimal to Binary

Problem: Convert non-negative integer `n` to binary string (no leading zeros except for n=0).

Input Format: Integer `n`.

Output Format: Binary string.

Constraints:

- `0 <= n <= 10^6`

Example:

```
Input:
10

Output:
1010
```

Explanation: 10 in binary is 1010.

Approach: Divide by 2, collect remainders, or use bit operations.

Q32. Binary to Decimal

Problem: Convert binary string `b` to decimal integer.

Input Format: Binary string.

Output Format: Decimal integer.

Constraints:

- String length up to 32

Example:

```
Input:
1010

Output:
10
```

Explanation: 1*2^3 + 0*2^2 + 1*2^1 + 0*2^0 = 10.

Approach: Iterate digits; multiply accumulated sum by 2 and add current digit.

Q33. Maximum Subarray Sum (Kadane's Algorithm)

Problem: Find the maximum sum of any contiguous subarray.

Input Format:

- Integer `n`
- `n` integers (may include negatives)

Output Format: Maximum sum.

Constraints:

- `1 <= n <= 10^4`

Example:

```
Input:
9
-2 1 -3 4 -1 2 1 -5 4

Output:
6
```

Explanation: Subarray [4, -1, 2, 1] has sum 6.

Approach: Kadane's algorithm: track max ending at each position.

Q34. Spiral Print of Matrix

Problem: Print matrix elements in spiral order (clockwise from top-left): right â†’ down â†’ left â†’ up, then inward.

Input Format:

- Integers `r` and `c`
- `r` rows of `c` integers each

Output Format: Elements in spiral order (space-separated).

Constraints:

- `1 <= r, c <= 50`

Example:

```
Input:
3 3
1 2 3
4 5 6
7 8 9

Output:
1 2 3 6 9 8 7 4 5
```

Explanation: Right (1,2,3), down (6,9), left (8,7), up (4), inward (5).

Approach: Track boundaries; traverse each boundary, shrink boundaries.

Q35. Bubble Sort

Problem: Sort an array using bubble sort.

Input Format:

- Integer `n`
- `n` integers

Output Format: Sorted array.

Constraints:

- `1 <= n <= 10^3`

Example:

```
Input:
5
5 1 4 2 8

Output:
1 2 4 5 8
```

Explanation: Repeatedly swap adjacent out-of-order elements.

Approach: Bubble sort: nested loops, swap if left > right.

Q36. Count Triangle Triplets

Problem: Given array `arr`, count triplets `(i, j, k)` with `0 <= i < j < k < n` where the three values form a valid triangle (satisfy triangle inequality).

Input Format:

- Integer `n`
- `n` integers

Output Format: Count of valid triplets.

Constraints:

- `1 <= n <= 500`

Example:

```
Input:
5
4 6 3 7 2

Output:
3
```

Explanation: Check combinations: (4,6,3), (4,3,7), (6,3,7) satisfy `a+b>c` for all permutations.

Approach: For each triplet, check if `a+b>c AND b+c>a AND c+a>b`.

Q37. Primes Below N (comma-separated)

Problem: Print all primes strictly less than `N`, separated by comma and space.

Input Format: Integer `N`.

Output Format: Comma-separated primes.

Constraints:

- `2 <= N <= 1000`

Example:

```
Input:
20

Output:
2, 3, 5, 7, 11, 13, 17, 19
```

Explanation: All primes < 20.

Approach: Sieve of Eratosthenes or primality test for each.

Q38. Sum of Digits

Problem: Compute sum of digits of a non-negative integer.

Input Format: Integer `n`.

Output Format: Sum of digits.

Constraints:

- `0 <= n <= 10^9`

Example:

```
Input:
1095

Output:
15
```

Explanation: 1 + 0 + 9 + 5 = 15.

Approach: Extract digits by modulo 10 and integer division.

Q39. Number Pattern (alternating increasing/decreasing)

Problem: Print `n` rows. Odd rows have numbers in increasing order; even rows in decreasing. Numbers are consecutive across rows.

Input Format: Integer `n`.

Output Format: `n` lines (numbers separated by `*`).

Constraints:

- `1 <= n <= 10`

Example:

```
Input:
4

Output:
1*2*3*4*17*18*19*20
5*6*7*14*15*16
8*9*12*13
10*11
```

Explanation: Consecutive number blocks; odd rows increase, even rows decrease within their block. (Note: exact pattern depends on specific problem variant.)

Approach: Track a global counter; for each row, print numbers in appropriate order.

Q40. Solid Square of Stars

Problem: Print an `n x n` solid square using `*`.

Input Format: Integer `n`.

Output Format: `n` lines, each with `n` stars.

Constraints:

- `1 <= n <= 20`

Example:

```
Input:
3

Output:
***
***
***
```

Explanation: Each of 3 rows contains 3 stars.

Approach: Nested loops; print n stars per line.

Q41. Maze Path (reachability)

Problem: Given a grid where 0 = wall, 1 = free path, 9 = cheese, and starting position (0,0), determine if mouse can reach any cell with value 9. Return 1 if reachable, else 0.

Input Format:

- Integers `r` and `c`
- `r` rows of `c` integers each

Output Format: 1 or 0.

Constraints:

- `1 <= r, c <= 10`

Example:

```
Input:
3 3
1 1 1
9 1 1
0 1 0

Output:
1
```

Explanation: Cheese is at (1, 0); reachable from (0, 0) via down movement.

Approach: BFS or DFS from (0, 0); visit all reachable free cells; check if any cell has value 9.

Q42. Distinct Elements from Two Arrays + Sum

Problem: Given two arrays, print all elements that appear in **exactly one** of them (not common). Also print their sum.

Input Format:

- Integer `n1`, then `n1` integers
- Integer `n2`, then `n2` integers

Output Format:

- Line 1: distinct elements (space-separated)
- Line 2: sum of those elements

Constraints:

- `1 <= n1, n2 <= 10^3`

Example:

```
Input:
5
1 2 3 4 5
4
2 6 8 10

Output:
1 3 4 5 6 8 10
37
```

Explanation: Common: {2}; Distinct: {1, 3, 4, 5, 6, 8, 10}; Sum = 37.

Approach: Use sets; find symmetric difference.

Q43. Swap Alternate Elements

Problem: Swap adjacent pairs in an array: swap (0,1), (2,3), (4,5), â€¦ If length is odd, last element stays.

Input Format:

- Integer `n`
- `n` integers

Output Format: Modified array.

Constraints:

- `1 <= n <= 10^3`

Example:

```
Input:
5
10 20 30 40 50

Output:
20 10 40 30 50
```

Explanation: Swap first two, next two; 50 stays.

Approach: Iterate with step 2; swap elements at i and i+1.

Q44. Print Pascal Pattern (concatenated, no spaces)

Problem: Print first `n` rows of Pascal's triangle as concatenated digits (valid for small n where no number exceeds single digit or two digits).

Input Format: Integer `n` .

Output Format: `n` lines.

Constraints:

- `1 <= n <= 5`

Example:

```
Input:
5

Output:
11
121
1331
14641
15101051
```

Explanation: Pascal rows concatenated without spaces: 1 1 â†’ 11, 1 2 1 â†’ 121, etc.

Approach: Generate Pascal rows; concatenate and print.

Q45. Alternating Edge Increment Pattern

Problem: For input `n`, print `n` lines. Odd-numbered lines (1, 3, 5, ...) print repeated digit ending with next digit. Even-numbered lines (2, 4, 6, ...) start with next digit then repeat current digit.

Input Format: Integer `n`.

Output Format: `n` lines.

Constraints:

- `1 <= n <= 10`

Example:

```
Input:
6

Output:
1111112
3222222
3333334
5444444
5555556
7666666
```

Explanation: Pattern alternates; digit increments every 2 rows.

Approach: Track digit; for each line, print based on odd/even line number.

Q46. 2*n-1 Rows Numeric Block Pattern

Problem: Print `2*n-1` rows: top half increases width from 1 to n; bottom half mirrors down to 1. Each row contains same digit repeated (width of row), with digit increasing in top half and decreasing in bottom.

Input Format: Integer `n`.

Output Format: `2*n-1` lines.

Constraints:

- `1 <= n <= 10`

Example:

```
Input:
4

Output:
3
44
555
6666
555
44
3
```

Explanation: For n=4, digits range 3 to 6; rows have widths 1,2,3,4,3,2,1.

Approach: Top half: print digit repeated i times for rows 1..n; bottom half: mirror.

Q47. Mixed Number-Star Pattern with Hyphen Indentation

Problem: Print `n` rows. Each row starts with some hyphens (increasing), then prints a number sequence separated by `*`. Outer numbers increase globally; right-side block is ordered oppositely. (Pattern is complex; see example.)

Input Format: Integer `n`.

Output Format: `n` lines.

Constraints:

- `1 <= n <= 5`

Example:

```
Input:
4

Output:
1*2*3*4*17*18*19*20
--5*6*7*14*15*16
----8*9*12*13
------10*11
```

Explanation: Each row consumes a range of consecutive numbers; left side increases, right side filled to maintain consecutive sequence; indentation increases.

Approach: Calculate ranges for each row; left half increments, right half decrements within the range.

Q48. Count Occurrences of Digit in Number

Problem: Given integer `num1` and single digit `num2` (0â€“9), count how many times `num2` appears in the decimal representation of `num1`.

Input Format: Two integers `num1` and `num2`.

Output Format: Count.

Constraints:

- `0 <= num1 <= 10^9`
- `0 <= num2 <= 9`

Example:

```
Input:
1222212 2

Output:
5
```

Explanation: Digit 2 appears 5 times in 1222212.

Approach: Convert to string and count occurrences, or use modulo arithmetic.

Q49. Primes Below N (comma-separated) â€” variant

Problem: Same as Q37. Print all primes below `N` comma-separated.

Input Format: Integer `N`.

Output Format: Comma-separated primes.

Example:

```
Input:
50

Output:
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
```

Approach: Sieve or primality check.

Q50. Three Dice Sum Outcomes

Problem: Given target sum `S`, count ordered triples `(a, b, c)` where each is 1â€“6 (standard die) and `a + b + c = S`.

Input Format: Integer `S`.

Output Format: Count.

Constraints:

- `3 <= S <= 18`

Example:

```
Input:
5

Output:
6
```

Explanation: Triples: (1,1,3), (1,2,2), (1,3,1), (2,1,2), (2,2,1), (3,1,1) â†' 6 outcomes.

Approach: Triple nested loop from 1 to 6; count matching.

Q51. Armstrong Numbers in Range

Problem: Print all Armstrong numbers in range `[L, R]` (inclusive) in ascending order.

Input Format: Two integers `L` and `R`.

Output Format: Armstrong numbers (space-separated).

Constraints:

- `1 <= L <= R <= 10000`

Example:

```
Input:
100 400

Output:
153 370 371
```

Explanation: Each is Armstrong (e.g., 153 = 1^3 + 5^3 + 3^3).

Approach: For each number in range, check if Armstrong.

Q52. Character Frequency (aâ€“z)

Problem: Given a lowercase string, count character frequencies and print `character count` (one per line) in alphabetical order; include only non-zero counts.

Input Format: Single lowercase string.

Output Format: Lines of `character count`.

Constraints:

- String length up to 10^3

Example:

```
Input:
helloworld

Output:
d 1
e 1
h 1
l 3
o 2
r 1
w 1
```

Explanation: Count each letter; print sorted.

Approach: Use frequency map; sort and print.

Q53. Check Palindrome Number (no string conversion)

Problem: Determine if integer `n` is a palindrome without converting to string. Print `YES` or `NO`.

Input Format: Integer `n`.

Output Format: `YES` or `NO`.

Constraints:

- `0 <= n <= 10^9`

Example:

```
Input:
12321

Output:
YES
```

Explanation: Reading left-to-right and right-to-left gives same sequence.

Approach: Extract digits via modulo; reverse and compare.

Q54. Reverse Each Word in Sentence

Problem: Reverse each word in a sentence individually; keep word order same. Maintain single space between words.

Input Format: One line (sentence with words separated by spaces).

Output Format: Transformed sentence.

Constraints:

- Line length up to 10^3

Example:

```
Input:
amcat automata test

Output:
tacma atamotua tset
```

Explanation: Reverse characters within each word.

Approach: Split by space; reverse each word; join.

Q55. Pythagorean Triplet Exists

Problem: Given array `arr`, return `True` if there exist indices `i < j < k` with `arr[i]^2 + arr[j]^2 = arr[k]^2` (in any assignment; i.e., some element serves as hypotenuse). Otherwise return `False`.

Input Format:

- Integer `n`
- `n` integers

Output Format: `True` or `False`.

Constraints:

- `1 <= n <= 100`

Example:

```
 Input:
5
3 1 4 6 5

Output:
True
```

Explanation: 3^2 + 4^2 = 5^2 (9 + 16 = 25), so Pythagorean triplet exists.

Approach: For each element as potential hypotenuse, check if two other elements' squares sum to it.